

C# and ADO.NET

Complete Solutions — 30 Exercises

This document provides complete, well-commented C# solutions for all 30 exercises covering .NET setup, OOP fundamentals, generics, LINQ, async/await, file I/O, threading, and ADO.NET database operations.

1. Set Up the Development Environment

Objective: Install .NET SDK, create and run a Hello World console app.

```
// File: Program.cs
// Commands to run in terminal:
// dotnet new console -n HelloWorld
// cd HelloWorld
// dotnet run

using System;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Hello, World!");
        Console.WriteLine($" .NET Version: {Environment.Version}");
        Console.WriteLine($"OS: {Environment.OSVersion}");
        Console.ReadKey();
    }
}
```

Key Points: 'dotnet new console' scaffolds a project. Program.cs is the entry point. Main() is the starting method. Console.WriteLine prints to stdout. Environment.Version confirms .NET SDK version installed.

2. Explore Value vs Reference Types

Objective: Demonstrate differences between value types (int, double) and reference types (string, class).

```
using System;

class MyClass { public int Value { get; set; } }

class Program
{
    // Value type: copy is passed – original unchanged
    static void ModifyValue(int x) { x = 999; }

    // Reference type: pointer is passed – original changed
    static void ModifyReference(MyClass obj) { obj.Value = 999; }

    // String is immutable reference type – acts like value type
    static void ModifyString(string s) { s = "changed"; }

    static void Main()
    {
        // Value type demo
        int num = 10;
        Console.WriteLine($"Before ModifyValue: {num}");
        ModifyValue(num);
        Console.WriteLine($"After ModifyValue: {num}"); // still 10

        // Reference type demo
    }
}
```

```

MyClass obj = new MyClass { Value = 10 };
Console.WriteLine($"Before ModifyReference: {obj.Value}");
ModifyReference(obj);
Console.WriteLine($"After ModifyReference: {obj.Value}"); // 999

// String demo (immutable)
string str = "original";
Console.WriteLine($"Before ModifyString: {str}");
ModifyString(str);
Console.WriteLine($"After ModifyString: {str}"); // still "original"
}
}

```

Key Points: Value types (int, double, bool, struct) are stored on the stack — copies are passed. Reference types (class, array, string) store a reference to heap memory. string is immutable: any 'modification' creates a new string object.

3. Use Primary Constructors in C# 12

Objective: Create a Person class using C# 12 primary constructor syntax.

```
using System;

// C# 12 Primary Constructor – parameters available throughout the class
public class Person(string firstName, string lastName, int age)
{
    // Auto-implemented properties initialized from primary constructor params
    public string FirstName { get; init; } = firstName;
    public string LastName { get; init; } = lastName;
    public int Age { get; init; } = age;

    // Computed property
    public string FullName => $"{FirstName} {LastName}";

    public void DisplayInfo()
    {
        Console.WriteLine($"Name: {FullName}");
        Console.WriteLine($"Age : {Age}");
        Console.WriteLine($"Adult: {(Age >= 18 ? "Yes" : "No")}");
    }
}

class Program
{
    static void Main()
    {
        var p1 = new Person("Alice", "Johnson", 30);
        p1.DisplayInfo();

        var p2 = new Person("Bob", "Smith", 17);
        p2.DisplayInfo();

        Console.WriteLine($"Full name: {p2.FullName}");
    }
}
```

Key Points: Primary constructors (C# 12) put parameters directly after the class name. Parameters are in scope throughout the class body. `init` means property can only be set at construction. `=>` lambda syntax creates a computed read-only property.

4. Demonstrate Type Inference with `var` and `new()`

Objective: Understand `var` and `new()` target-typed expressions.

```
using System;
using System.Collections.Generic;

class Product { public string Name { get; set; } public double Price { get; set; } }

class Program
{
    static void Main()
    {
        // var: compiler infers the type from RHS
        var age = 25; // int
        var name = "Alice"; // string
        var price = 99.99; // double
        var isOpen = true; // bool

        Console.WriteLine($"age : {age.GetType().Name} = {age}");
        Console.WriteLine($"name : {name.GetType().Name} = {name}");
        Console.WriteLine($"price : {price.GetType().Name} = {price}");
        Console.WriteLine($"isOpen: {isOpen.GetType().Name} = {isOpen}");

        // new() target-typed (C# 9+): type inferred from LHS
    }
}
```

```

List<string> cities    = new();           // same as new List<string>()
Product    product    = new() { Name = "Laptop", Price = 45000 };
Dictionary<int,string> map = new();

cities.Add("Hyderabad");
cities.Add("Bengaluru");
Console.WriteLine($"Cities: {string.Join(", ", cities)}");
Console.WriteLine($"Product: {product.Name} @ {product.Price}");

// When var is NOT ideal (reduces readability)
var result = GetData(); // reader must look up GetData() to know type
Console.WriteLine(result);
}

static string GetData() => "Sample Data";
}

```

Key Points: var is resolved at compile time — it's still strongly typed. new() (C# 9+) infers the type from the left-hand side declaration. Use var when the type is obvious from the RHS; avoid it when it hides the type.

5. Perform Conditional Logic for Grade Calculation

Objective: Use if/else, switch with pattern matching for grade calculation.

```
using System;

class Program
{
    static string GetGradeIfElse(int score)
    {
        if (score >= 90) return "A - Excellent";
        else if (score >= 80) return "B - Very Good";
        else if (score >= 70) return "C - Good";
        else if (score >= 60) return "D - Pass";
        else return "F - Fail";
    }

    // C# pattern matching switch expression (C# 8+)
    static string GetGradeSwitch(int score) => score switch
    {
        >= 90 => "A - Excellent",
        >= 80 => "B - Very Good",
        >= 70 => "C - Good",
        >= 60 => "D - Pass",
        _ => "F - Fail" // default case
    };

    static void Main()
    {
        Console.Write("Enter score (0-100): ");
        if (int.TryParse(Console.ReadLine(), out int score))
        {
            Console.WriteLine($"if/else : {GetGradeIfElse(score)}");
            Console.WriteLine($"switch : {GetGradeSwitch(score)}");
        }
        else
        {
            Console.WriteLine("Invalid input.");
        }
    }
}
```

Key Points: Switch expressions (C# 8+) are more concise than switch statements. >= pattern matching replaces range comparisons elegantly. _ is the discard/default case in switch expressions. int.TryParse safely converts string input without throwing exceptions.

6. Loop Through an Array with Different Loop Types

Objective: Iterate arrays using for, foreach, while, and do-while loops.

```
using System;

class Program
{
    static void Main()
    {
        int[] numbers = { 5, 12, 3, 8, 21, 7, 15 };

        // 1. for loop – skip even numbers
        Console.WriteLine("for loop (odd only):");
        for (int i = 0; i < numbers.Length; i++)
        {
            if (numbers[i] % 2 == 0) continue; // skip even
            Console.Write($" [{i}]= {numbers[i]}");
        }

        // 2. foreach – stop at value > 15
        Console.WriteLine("
    
```

```

foreach (stop at >15:");
    foreach (int n in numbers)
    {
        if (n > 15) break;
        Console.Write($" {n}");
    }

    // 3. while loop
    Console.WriteLine("
while loop:");
    int idx = 0;
    while (idx < numbers.Length)
    {
        Console.Write($" {numbers[idx]}");
        idx++;
    }

    // 4. do-while – always executes at least once
    Console.WriteLine("
do-while loop:");
    int j = 0;
    do
    {
        Console.Write($" {numbers[j]}");
        j++;
    } while (j < numbers.Length);

    Console.WriteLine();
}
}

```

Key Points: for is best when you need the index. foreach is cleaner for simple iteration. while checks condition before each iteration. do-while always runs the body at least once. continue skips the current iteration; break exits the loop entirely.

7. Implement Method Overloading

Objective: Define CalculateTotal with different parameter types/counts.

```
using System;

class Calculator
{
    // Overload 1: two integers
    public static int CalculateTotal(int a, int b)
    {
        Console.WriteLine($"Overload: int + int");
        return a + b;
    }

    // Overload 2: three doubles
    public static double CalculateTotal(double a, double b, double c)
    {
        Console.WriteLine($"Overload: double x3");
        return a + b + c;
    }

    // Overload 3: array of decimals
    public static decimal CalculateTotal(params decimal[] values)
    {
        Console.WriteLine($"Overload: params decimal[]");
        decimal sum = 0;
        foreach (var v in values) sum += v;
        return sum;
    }

    // Overload 4: string (parse and sum)
    public static double CalculateTotal(string a, string b)
    {
        Console.WriteLine($"Overload: string + string (parsed)");
        return double.Parse(a) + double.Parse(b);
    }
}

class Program
{
    static void Main()
    {
        Console.WriteLine(Calculator.CalculateTotal(10, 20));
        Console.WriteLine(Calculator.CalculateTotal(1.5, 2.5, 3.0));
        Console.WriteLine(Calculator.CalculateTotal(100m, 200m, 300m, 400m));
        Console.WriteLine(Calculator.CalculateTotal("45.5", "54.5"));
    }
}
```

Key Points: Overloading = same method name, different signatures (parameter types/count). Compiler resolves the correct overload at compile time. params keyword allows variable-length argument lists. Return type alone cannot distinguish overloads.

8. Use ref, out, and in Parameters

Objective: Understand ref (read/write), out (write-only), in (read-only) modifiers.

```
using System;

class Program
{
    // ref: must be initialized before call; can read and modify
    static void DoubleValue(ref int x)
    {
        x = x * 2;
    }
}
```

```

// out: does not need initialization; MUST be assigned in method
static void GetMinMax(int[] arr, out int min, out int max)
{
    min = arr[0]; max = arr[0];
    foreach (int n in arr)
    {
        if (n < min) min = n;
        if (n > max) max = n;
    }
}

// in: read-only reference (no copy made, cannot modify)
static double ComputeArea(in double radius)
{
    // radius = 10; // ERROR: in parameter is read-only
    return Math.PI * radius * radius;
}

static void Main()
{
    // ref demo
    int val = 5;
    Console.WriteLine($"Before ref: {val}");
    DoubleValue(ref val);
    Console.WriteLine($"After  ref: {val}"); // 10

    // out demo
    int[] nums = { 4, 1, 9, 2, 7 };
    GetMinMax(nums, out int min, out int max);
    Console.WriteLine($"Min={min}, Max={max}");

    // in demo
    double r = 7.5;
    Console.WriteLine($"Area = {ComputeArea(in r):F2}");
}
}

```

Key Points: ref — caller initializes, method can read & write. out — method must assign before returning; caller doesn't need to initialize. in — passes by reference for performance but prevents modification (useful for large structs).

9. Use Local Functions

Objective: Define and use a local function inside CalculateFactorial.

```
using System;

class Program
{
    static long CalculateFactorial(int n)
    {
        // Local function: only visible inside CalculateFactorial
        long Factorial(int num)
        {
            if (num <= 1) return 1;
            return num * Factorial(num - 1); // recursive call
        }

        if (n < 0)
            throw new ArgumentException("Factorial not defined for negatives.");

        return Factorial(n);
    }

    static void Main()
    {
        for (int i = 0; i <= 10; i++)
        {
            Console.WriteLine($"{i}! = {CalculateFactorial(i)}");
        }
    }
}
```

Key Points: Local functions are defined inside another method — they're invisible outside it. They can be recursive and can capture variables from the enclosing scope (closures). Useful to decompose complex logic without polluting the class namespace.

10. Demonstrate OOP Basics with Constructors

Objective: Create a Car class with default and parameterized constructors.

```
using System;

class Car
{
    public string Make { get; set; }
    public string Model { get; set; }
    public int Year { get; set; }

    // Default constructor
    public Car()
    {
        Make = "Unknown";
        Model = "Unknown";
        Year = 2000;
    }

    // Parameterized constructor
    public Car(string make, string model, int year)
    {
        Make = make;
        Model = model;
        Year = year;
    }

    // Constructor chaining using this()
    public Car(string make, string model) : this(make, model, 2025) { }

    public void DisplayInfo()
```

```

        => Console.WriteLine($"{Year} {Make} {Model}");
    }

class Program
{
    static void Main()
    {
        Car c1 = new Car();           // default
        Car c2 = new Car("Toyota", "Camry", 2023); // parameterized
        Car c3 = new Car("Honda", "City");         // chained (year=2025)

        c1.DisplayInfo();
        c2.DisplayInfo();
        c3.DisplayInfo();
    }
}

```

Key Points: Default constructor runs when no args are given. Parameterized constructor sets values at creation. Constructor chaining (: this(...)) reuses another constructor to avoid duplicating code. => expression body syntax shortens simple method definitions.

11. Demonstrate Access Modifiers

Objective: Show public, private, and protected access levels.

```
using System;

class BankAccount
{
    public    string AccountHolder { get; set; }    // accessible everywhere
    private  double _balance;                    // only inside BankAccount
    protected string AccountType    = "Savings";    // BankAccount + subclasses

    public BankAccount(string holder, double initial)
    {
        AccountHolder = holder;
        _balance = initial;
    }

    public void Deposit(double amount)
    {
        if (amount > 0) _balance += amount;
    }

    public double GetBalance() => _balance;    // controlled access to private field
}

class PremiumAccount : BankAccount
{
    public PremiumAccount(string holder, double initial)
        : base(holder, initial) { }

    public void ShowType()
    {
        // Can access protected member from base class
        Console.WriteLine($"Account type: {AccountType}");
        // Console.WriteLine(_balance); // ERROR: private not accessible
    }
}

class Program
{
    static void Main()
    {
        var acct = new BankAccount("Alice", 5000);
        acct.Deposit(1500);
        Console.WriteLine($"Holder : {acct.AccountHolder}");
        Console.WriteLine($"Balance: {acct.GetBalance()}");
        // acct._balance = 0; // ERROR: private

        var premium = new PremiumAccount("Bob", 10000);
        premium.ShowType();
    }
}
```

Key Points: public — accessible from anywhere. private — only within the same class. protected — within the class and its derived classes. Encapsulation: expose private data through public methods (GetBalance) to control access.

12. Use Auto-Properties and Backing Fields

Objective: Auto-properties for Name; backing field with validation for Price.

```
using System;

class Product
{
    // Auto-implemented property – compiler generates the backing field
    public string Name { get; set; }
```

```

// Backing field with validation
private double _price;
public double Price
{
    get => _price;
    set
    {
        if (value < 0)
            throw new ArgumentException("Price cannot be negative.");
        _price = value;
    }
}

public string Category { get; init; } // init-only (set once at construction)

public Product(string name, double price, string category)
{
    Name      = name;
    Price     = price;    // goes through setter validation
    Category  = category;
}

public override string ToString()
    => $"Product: {Name} | Price: {Price:C} | Category: {Category}";
}

class Program
{
    static void Main()
    {
        var p = new Product("Laptop", 45000, "Electronics");
        Console.WriteLine(p);

        p.Price = 48000;    // valid update
        Console.WriteLine($"Updated Price: {p.Price:C}");

        try
        {
            p.Price = -500; // triggers validation
        }
        catch (ArgumentException ex)
        {
            Console.WriteLine($"Error: {ex.Message}");
        }
    }
}

```

Key Points: Auto-property { get; set; } lets compiler generate backing field automatically. Manual backing field with validation in setter enforces business rules. init accessor (C# 9+) allows setting only during object initialization. :C format specifier formats as currency.

13. Create and Use Records with init Properties

Objective: Define an immutable Employee record; use 'with' to create modified copies.

```
using System;

// Record: value-based equality, immutable by default
public record Employee(
    int EmployeeId,
    string Name,
    string Department,
    double Salary
);

// Record with extra computed member
public record Product2
{
    public required string Name { get; init; }
    public required decimal Price { get; init; }
    public string Display => $"{Name}: {Price:C}";
}

class Program
{
    static void Main()
    {
        var emp1 = new Employee(1, "Alice", "Engineering", 75000);
        Console.WriteLine($"Original : {emp1}");

        // 'with' expression: creates a copy with specified changes
        var emp2 = emp1 with { Salary = 85000, Department = "Management" };
        Console.WriteLine($"Modified : {emp2}");
        Console.WriteLine($"Original unchanged: {emp1}");

        // Records use value-based equality
        var emp3 = new Employee(1, "Alice", "Engineering", 75000);
        Console.WriteLine($"emp1 == emp3: {emp1 == emp3}"); // True

        // Product2 using required init
        var prod = new Product2 { Name = "Tablet", Price = 25000 };
        Console.WriteLine(prod.Display);
    }
}
```

Key Points: record creates an immutable reference type with auto-generated ToString, Equals, GetHashCode, and deconstruction. 'with' creates a shallow copy with selected overrides. The original is never modified. required (C# 11+) enforces property initialization.

14. Demonstrate Inheritance and Method Overriding

Objective: Base class Shape with virtual Draw(); derived Circle and Rectangle override it.

```
using System;

abstract class Shape
{
    public string Color { get; set; } = "Black";

    // virtual: can be overridden in derived classes
    public virtual void Draw()
        => Console.WriteLine($"Drawing a {Color} shape.");

    // abstract: MUST be overridden
    public abstract double Area();
}

class Circle : Shape
{
}
```

```

    public double Radius { get; set; }
    public Circle(double r, string color) { Radius = r; Color = color; }

    public override void Draw()
        => Console.WriteLine($"Drawing a {Color} Circle (r={Radius})");

    public override double Area()
        => Math.PI * Radius * Radius;
}

class Rectangle : Shape
{
    public double Width { get; set; }
    public double Height { get; set; }
    public Rectangle(double w, double h) { Width = w; Height = h; }

    public override void Draw()
        => Console.WriteLine($"Drawing Rectangle {Width}x{Height}");

    public override double Area() => Width * Height;
}

class Program
{
    static void Main()
    {
        Shape[] shapes = { new Circle(5, "Red"), new Rectangle(4, 6) };

        foreach (Shape s in shapes)
        {
            s.Draw(); // polymorphic dispatch
            Console.WriteLine($"Area: {s.Area():F2}");
        }
    }
}

```

Key Points: virtual allows optional override; abstract forces it. override keyword replaces base implementation.
Polymorphism: calling s.Draw() on a Shape[] calls the correct derived implementation at runtime.

15. Differentiate Abstract Classes and Interfaces

Objective: Abstract class Vehicle with Drive(); interface IDrivable with Start(). Car implements both.

```
using System;

// Abstract class: can have implementations + state
abstract class Vehicle
{
    public string Brand { get; set; }
    public Vehicle(string brand) { Brand = brand; }

    public abstract void Drive(); // must override

    public virtual void FuelUp()
        => Console.WriteLine($"{Brand} is fueling up.");
}

// Interface: pure contract, no state (pre-C# 8)
interface IDrivable
{
    void Start();
    void Stop();
    string GetStatus(); // interface method (no body)
}

interface IElectric
{
    void Charge();
}

// Class can inherit ONE abstract class but MANY interfaces
class Car : Vehicle, IDrivable, IElectric
{
    private bool _running = false;

    public Car(string brand) : base(brand) { }

    public override void Drive()
        => Console.WriteLine($"{Brand} Car is driving.");

    public void Start() { _running = true; Console.WriteLine($"{Brand} started."); }
    public void Stop() { _running = false; Console.WriteLine($"{Brand} stopped."); }
    public string GetStatus() => _running ? "Running" : "Stopped";
    public void Charge() => Console.WriteLine($"{Brand} EV is charging.");
}

class Program
{
    static void Main()
    {
        Car car = new Car("Tesla");
        car.Start();
        car.Drive();
        car.FuelUp();
        car.Charge();
        Console.WriteLine($"Status: {car.GetStatus()}");
        car.Stop();
        Console.WriteLine($"Status: {car.GetStatus()}");

        // Polymorphism through interface
        IDrivable d = new Car("BMW");
        d.Start();
        d.Stop();
    }
}
```

Key Points: Abstract class: single inheritance, can have fields/constructors/implemented methods. Interface: multiple implementation, pure contract. A class can only extend one abstract class but implement many interfaces.

16. Handle Null References Safely

Objective: Use ?. (null-conditional) and ?? (null-coalescing) operators.

```
using System;

class Address
{
    public string City    { get; set; }
    public string Country { get; set; }
}

class Person2
{
    public string Name    { get; set; }
    public Address Address { get; set; } // nullable reference
}

class Program
{
    static void Main()
    {
        Person2 person1 = new Person2 { Name = "Alice",
            Address = new Address { City = "Hyderabad", Country = "India" } };

        Person2 person2 = new Person2 { Name = "Bob", Address = null };
        Person2 person3 = null;

        // ?. – returns null instead of throwing NullReferenceException
        Console.WriteLine(person1?.Address?.City);    // "Hyderabad"
        Console.WriteLine(person2?.Address?.City);    // null (no exception)
        Console.WriteLine(person3?.Name);             // null (no exception)

        // ?? – returns right side if left side is null
        string city = person2?.Address?.City ?? "City Unknown";
        Console.WriteLine(city); // "City Unknown"

        // ??= – assign only if left side is null (C# 8+)
        person2.Address ??= new Address { City = "Chennai", Country = "India" };
        Console.WriteLine(person2.Address.City); // "Chennai"

        // Null check the traditional way for comparison
        string name = (person3 != null) ? person3.Name : "Anonymous";
        // Equivalent with ??:
        string name2 = person3?.Name ?? "Anonymous";
        Console.WriteLine(name2);
    }
}
```

Key Points: ?. (null-conditional): evaluates to null instead of throwing if object is null. ?? (null-coalescing): returns right operand when left is null. ??= (null-coalescing assignment, C# 8+): assigns only if variable is null.

17. Use Null-Conditional Chaining in a Contact App

Objective: Safely access contact properties using ?. chaining.

```
using System;
using System.Collections.Generic;

class Contact
{
    public string Name      { get; set; }
    public string PhoneNumber { get; set; }
    public string Email      { get; set; }
}

class ContactApp
{
    private List<Contact> _contacts = new();

    public void AddContact(Contact c) => _contacts.Add(c);

    public void DisplayContact(int index)
    {
        Contact c = (index >= 0 && index < _contacts.Count)
            ? _contacts[index] : null;

        // Safe chained access
        string displayName = c?.Name      ?? "No Name";
        string displayPhone = c?.PhoneNumber ?? "No Phone";
        string displayEmail = c?.Email      ?? "No Email";

        Console.WriteLine($"Name : {displayName}");
        Console.WriteLine($"Phone: {displayPhone}");
        Console.WriteLine($"Email: {displayEmail}");
    }
}

class Program
{
    static void Main()
    {
        var app = new ContactApp();
        app.AddContact(new Contact { Name = "Alice", PhoneNumber = "9876543210",
                                     Email = "alice@example.com" });
        app.AddContact(new Contact { Name = "Bob", PhoneNumber = null });

        Console.WriteLine("--- Contact 0 ---");
        app.DisplayContact(0);
        Console.WriteLine("--- Contact 1 (missing email) ---");
        app.DisplayContact(1);
        Console.WriteLine("--- Contact 99 (not found) ---");
        app.DisplayContact(99);
    }
}
```

Key Points: ?. chains safely — if any link is null the whole chain returns null. Combining ?. and ?? provides safe access with sensible defaults. This pattern is common in UI code where data may be partially filled.

18. Use the required Modifier in C# 12

Objective: Enforce required properties with the required modifier.

```
using System;

class Student
{
    // required: compiler error if not set in object initializer
    public required int    StudentId { get; init; }
    public required string Name      { get; init; }
```

```

    public required string Course    { get; init; }
    public int    Age    { get; init; }    // optional
    public string Email    { get; set; }    // optional, mutable

    public override string ToString()
        => $"[{StudentId}] {Name} | Course: {Course} | Age: {Age}";
}

class Program
{
    static void Main()
    {
        // Valid: all required properties provided
        var s1 = new Student
        {
            StudentId = 101,
            Name       = "Alice Johnson",
            Course     = "Computer Science",
            Age        = 20,
            Email      = "alice@uni.edu"
        };
        Console.WriteLine(s1);

        // The following would cause a COMPILE-TIME ERROR:
        // var s2 = new Student { StudentId = 102, Name = "Bob" };
        // Error: Required member 'Student.Course' must be set.

        Console.WriteLine("required modifier enforces initialization at compile time!");
    }
}

```

Key Points: required (C# 11+) forces callers to set the property in the object initializer. Unlike constructor parameters, required works with object initializers. Combined with init, it creates properties that must be set exactly once at construction.

19. Work with Lists and Dictionaries

Objective: Use List and Dictionary — add, iterate, remove.

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        // --- List<string> ---
        List<string> cities = new List<string> { "Hyderabad", "Mumbai", "Delhi" };
        cities.Add("Bengaluru");
        cities.Insert(1, "Chennai");    // insert at index 1

        Console.WriteLine("Cities:");
        foreach (string city in cities)
            Console.WriteLine($" {city}");

        cities.Remove("Mumbai");        // remove by value
        cities.RemoveAt(0);             // remove by index
        Console.WriteLine($"After removals: {string.Join(", ", cities)}");
        Console.WriteLine($"Contains Delhi: {cities.Contains("Delhi")}");

        // --- Dictionary<int, string> ---
        Dictionary<int, string> employees = new()
        {
            { 1, "Alice Johnson" },
            { 2, "Bob Smith" },
            { 3, "Charlie Lee" }
        };

        employees[4] = "Diana King";    // add or update
        employees.Add(5, "Ethan Hunt");

        Console.WriteLine("Employees:");
        foreach (var kvp in employees)
            Console.WriteLine($" ID {kvp.Key}: {kvp.Value}");

        // Safe lookup
        if (employees.TryGetValue(3, out string name))
            Console.WriteLine($"Found: {name}");

        employees.Remove(2);
        Console.WriteLine($"Count after remove: {employees.Count}");
    }
}
```

Key Points: List is a dynamic array. Dictionary stores key-value pairs. TryGetValue is safer than [] for lookups (no KeyNotFoundException). foreach with KeyValuePair iterates dictionary entries. string.Join formats list output.

20. Use LINQ for Filtering and Projection

Objective: Filter and project Order objects using LINQ.

```
using System;
using System.Collections.Generic;
using System.Linq;

class Order
{
    public int    OrderId    { get; set; }
    public string CustomerName { get; set; }
    public double TotalAmount { get; set; }
    public string Status     { get; set; }
}
```

```

class Program
{
    static void Main()
    {
        var orders = new List<Order>
        {
            new Order { OrderId=1, CustomerName="Alice", TotalAmount=1500, Status="Completed" },
            new Order { OrderId=2, CustomerName="Bob", TotalAmount=850, Status="Pending" },
            new Order { OrderId=3, CustomerName="Charlie", TotalAmount=3200, Status="Completed" },
            new Order { OrderId=4, CustomerName="Diana", TotalAmount=450, Status="Cancelled" },
            new Order { OrderId=5, CustomerName="Ethan", TotalAmount=2100, Status="Pending" },
        };

        // Filter + Order + Project using LINQ method syntax
        var highValueCompleted = orders
            .Where(o => o.TotalAmount > 1000 && o.Status == "Completed")
            .OrderByDescending(o => o.TotalAmount)
            .Select(o => new { o.OrderId, o.CustomerName,
                Amount = $"Rs.{o.TotalAmount:N0}" });

        Console.WriteLine("High-value Completed Orders:");
        foreach (var o in highValueCompleted)
            Console.WriteLine($" #{o.OrderId} {o.CustomerName}: {o.Amount}");

        // Aggregations
        Console.WriteLine($"
Total Revenue : Rs.{orders.Sum(o => o.TotalAmount):N0}");
        Console.WriteLine($"Average Order : Rs.{orders.Average(o => o.TotalAmount):N0}");
        Console.WriteLine($"Pending Count : {orders.Count(o => o.Status == "Pending")}");
        Console.WriteLine($"Max Order : Rs.{orders.Max(o => o.TotalAmount):N0}");

        // GroupBy
        var byStatus = orders.GroupBy(o => o.Status);
        Console.WriteLine("
Orders by Status:");
        foreach (var grp in byStatus)
            Console.WriteLine($" {grp.Key}: {grp.Count()} orders");
    }
}

```

Key Points: Where() filters, Select() projects, OrderBy/OrderByDescending sorts. Anonymous types (new { }) create lightweight projection objects. LINQ aggregates: Sum, Average, Count, Max, Min. GroupBy partitions data into groups.

21. Use Pattern Matching with is and switch

Objective: Type-check objects using is and perform operations with enhanced switch.

```
using System;

class Animal { public string Name { get; set; } }
class Dog : Animal { public string Breed { get; set; } }
class Cat : Animal { public bool IsIndoor { get; set; } }
class Bird : Animal { public double WingSpan { get; set; } }

class Program
{
    static void DescribeAnimal(object obj)
    {
        // is pattern matching with variable declaration
        if (obj is Dog d)
            Console.WriteLine($"Dog: {d.Name}, Breed: {d.Breed}");
        else if (obj is Cat c)
            Console.WriteLine($"Cat: {c.Name}, Indoor: {c.IsIndoor}");
        else if (obj is string s)
            Console.WriteLine($"String of length {s.Length}: {s}");
        else if (obj is int i when i > 0)
            Console.WriteLine($"Positive int: {i}");
        else
            Console.WriteLine($"Unknown: {obj}");
    }

    static string ClassifyAnimal(Animal a) => a switch
    {
        Dog { Breed: "Labrador" } d => $"{d.Name} is a friendly Labrador",
        Dog d => $"{d.Name} is a dog ({d.Breed})",
        Cat { IsIndoor: true } c => $"{c.Name} is an indoor cat",
        Cat c => $"{c.Name} is an outdoor cat",
        Bird { WingSpan: > 1.0 } b => $"{b.Name} has a large wingspan",
        _ => $"Unknown animal: {a.Name}"
    };

    static void Main()
    {
        DescribeAnimal(new Dog { Name = "Rex", Breed = "German Shepherd" });
        DescribeAnimal(new Cat { Name = "Whiskers", IsIndoor = true });
        DescribeAnimal("Hello Pattern Matching");
        DescribeAnimal(42);

        Animal[] animals = {
            new Dog { Name="Buddy", Breed="Labrador" },
            new Cat { Name="Luna", IsIndoor=false },
            new Bird { Name="Tweety", WingSpan=1.5 }
        };

        foreach (var a in animals)
            Console.WriteLine(ClassifyAnimal(a));
    }
}
```

Key Points: is declares a pattern variable (if (obj is Dog d)) — cleaner than cast+check. when adds a guard clause. Switch expressions with property patterns match specific values. Relational patterns (> 1.0) directly compare property values.

22. Create and Deconstruct Tuples

Objective: Return a tuple from a method and deconstruct it.

```
using System;

class Program
{
    // Method returning a named tuple
```

```

static (int Count, double Average, double Max) AnalyzeScores(int[] scores)
{
    int    count = scores.Length;
    double avg    = 0, max = double.MinValue;
    foreach (int s in scores) { avg += s; if (s > max) max = s; }
    return (count, avg / count, max);
}

// Method returning unnamed tuple
static (string, int) GetPersonInfo()
    => ("Alice Johnson", 30);

static void Main()
{
    int[] scores = { 85, 92, 78, 95, 88, 72, 90 };

    // Named tuple deconstruction
    var (count, avg, max) = AnalyzeScores(scores);
    Console.WriteLine($"Count   : {count}");
    Console.WriteLine($"Average: {avg:F2}");
    Console.WriteLine($"Max     : {max}");

    // Unnamed tuple – access by .Item1, .Item2
    var info = GetPersonInfo();
    Console.WriteLine($"
{info.Item1} is {info.Item2} years old.");

    // Deconstruct unnamed tuple
    var (personName, personAge) = GetPersonInfo();
    Console.WriteLine($"{{personName}} is {{personAge}} years old.");

    // Inline tuple without method
    (string City, int Population) cityInfo = ("Hyderabad", 9000000);
    Console.WriteLine($"
{cityInfo.City}: {cityInfo.Population:N0}");

    // Swap using tuple (no temp variable needed)
    int a = 10, b = 20;
    (a, b) = (b, a);
    Console.WriteLine($"After swap: a={{a}}, b={{b}}");
}
}

```

Key Points: Named tuple elements (Count, Average) make the return value self-documenting. Deconstruction assigns tuple elements to individual variables in one line. Tuple swap (a, b) = (b, a) is elegant. Tuples avoid creating a class just to return multiple values.

23. Simulate Async File Upload with Exception Handling

Objective: Async method simulating file upload with try-catch and Task.

```
using System;
using System.Threading.Tasks;

class FileUploader
{
    public async Task<string> UploadFileAsync(string fileName, long fileSizeBytes)
    {
        Console.WriteLine($"Starting upload: {fileName} ({fileSizeBytes/1024} KB)");

        try
        {
            if (string.IsNullOrEmpty(fileName))
                throw new ArgumentException("File name cannot be empty.");

            if (fileSizeBytes > 10 * 1024 * 1024) // 10 MB limit
                throw new InvalidOperationException("File exceeds 10MB limit.");

            // Simulate upload time proportional to file size
            int delayMs = (int)(fileSizeBytes / 1024); // 1ms per KB
            await Task.Delay(Math.Min(delayMs, 3000)); // cap at 3 seconds

            return $"SUCCESS: {fileName} uploaded at {DateTime.Now:HH:mm:ss}";
        }
        catch (ArgumentException ex)
        {
            return $"VALIDATION ERROR: {ex.Message}";
        }
        catch (InvalidOperationException ex)
        {
            return $"SIZE ERROR: {ex.Message}";
        }
        catch (Exception ex)
        {
            return $"UNEXPECTED ERROR: {ex.Message}";
        }
    }
}

class Program
{
    static async Task Main()
    {
        var uploader = new FileUploader();

        // Run multiple uploads concurrently
        var tasks = new[]
        {
            uploader.UploadFileAsync("report.pdf", 512 * 1024), // 512 KB
            uploader.UploadFileAsync("photo.jpg", 2 * 1024 * 1024), // 2 MB
            uploader.UploadFileAsync("", 100), // empty name
            uploader.UploadFileAsync("video.mp4", 15 * 1024 * 1024) // 15MB too big
        };

        string[] results = await Task.WhenAll(tasks);

        foreach (string result in results)
            Console.WriteLine(result);
    }
}
```

Key Points: async methods return Task. await suspends execution without blocking the thread. Task.WhenAll runs tasks concurrently and waits for all. try-catch inside async works exactly like synchronous code.

24. Serialize and Deserialize JSON Files

Objective: Serialize a User object to JSON and deserialize it back.

```
using System;
using System.IO;
using System.Text.Json;
using System.Text.Json.Serialization;

class User
{
    [JsonPropertyName("user_id")]
    public int    UserId { get; set; }

    [JsonPropertyName("name")]
    public string Name   { get; set; }

    [JsonPropertyName("age")]
    public int    Age    { get; set; }

    [JsonPropertyName("email")]
    public string Email  { get; set; }
}

class Program
{
    static void Main()
    {
        // Create object
        var user = new User { UserId = 1, Name = "Alice Johnson",
                             Age = 30, Email = "alice@example.com" };

        // Serialize to JSON string
        var options = new JsonSerializerOptions { WriteIndented = true };
        string json = JsonSerializer.Serialize(user, options);
        Console.WriteLine("Serialized JSON:");
        Console.WriteLine(json);

        // Save to file
        string filePath = "user.json";
        File.WriteAllText(filePath, json);
        Console.WriteLine($"
Saved to: {filePath}");

        // Read from file and deserialize
        string fileContent = File.ReadAllText(filePath);
        User deserialized = JsonSerializer.Deserialize<User>(fileContent);

        Console.WriteLine("
Deserialized Object:");
        Console.WriteLine($" ID    : {deserialized.UserId}");
        Console.WriteLine($" Name  : {deserialized.Name}");
        Console.WriteLine($" Age   : {deserialized.Age}");
        Console.WriteLine($" Email: {deserialized.Email}");

        // Cleanup
        File.Delete(filePath);
    }
}
```

Key Points: System.Text.Json is the built-in JSON library (.NET 5+). [JsonPropertyName] maps C# property names to JSON keys. WriteIndented makes output human-readable. File.WriteAllText/ReadAllText simplify file I/O for text content.

25. Use FileStream and MemoryStream

Objective: Read text using FileStream; write to MemoryStream.

```
using System;
using System.IO;
using System.Text;

class Program
{
    static void Main()
    {
        string filePath = "sample.txt";

        // Write sample file first
        File.WriteAllText(filePath, "Hello from FileStream!\nLine 2\nLine 3");

        // --- FileStream: read file bytes ---
        Console.WriteLine("=== FileStream Read ===");
        using (FileStream fs = new FileStream(filePath, FileMode.Open, FileAccess.Read))
        {
            byte[] buffer = new byte[fs.Length];
            int bytesRead = fs.Read(buffer, 0, buffer.Length);
            string content = Encoding.UTF8.GetString(buffer);
            Console.WriteLine($"Bytes read: {bytesRead}");
            Console.WriteLine($"Content:\n{content}");
        }

        // --- MemoryStream: in-memory byte operations ---
        Console.WriteLine("\n=== MemoryStream Write ===");
        using (MemoryStream ms = new MemoryStream())
        {
            string data = "Community Event Portal – In-Memory Data";
            byte[] bytes = Encoding.UTF8.GetBytes(data);

            ms.Write(bytes, 0, bytes.Length);
            Console.WriteLine($"Bytes written to MemoryStream: {ms.Length}");

            // Read back from MemoryStream
            ms.Position = 0; // rewind to beginning
            using (StreamReader sr = new StreamReader(ms))
            {
                Console.WriteLine($"Read back: {sr.ReadToEnd()}");
            }
        }

        File.Delete(filePath);
    }
}
```

Key Points: FileStream reads/writes raw bytes from disk files. MemoryStream works entirely in RAM — useful for temporary buffers, testing, compression. using statement ensures streams are disposed (closed) even if exceptions occur. Encoding.UTF8 converts between bytes and strings.

26. Demonstrate Race Conditions with Multi-threading

Objective: Show race condition on shared counter; fix with lock.

```
using System;
using System.Threading;

class Counter
{
    private int _count = 0;
    private readonly object _lockObj = new object();

    // UNSAFE: race condition – threads may read/write simultaneously
    public void IncrementUnsafe() => _count++;
}
```

```

// SAFE: lock ensures only one thread modifies at a time
public void IncrementSafe()
{
    lock (_lockObj) { _count++; }
}

public int Value => _count;
public void Reset() => _count = 0;
}

class Program
{
    static void RunTest(string label, Action incrementAction, Counter counter)
    {
        counter.Reset();
        Thread[] threads = new Thread[10];
        for (int i = 0; i < 10; i++)
        {
            threads[i] = new Thread(() =>
            {
                for (int j = 0; j < 1000; j++)
                    incrementAction();
            });
        }
        foreach (var t in threads) t.Start();
        foreach (var t in threads) t.Join();

        Console.WriteLine($"{label}: Expected=10000, Got={counter.Value}");
    }

    static void Main()
    {
        var counter = new Counter();

        // Race condition: result is unpredictable (< 10000 usually)
        RunTest("Unsafe (race)", counter.IncrementUnsafe, counter);

        // Thread-safe: always 10000
        RunTest("Safe (locked)", counter.IncrementSafe, counter);
    }
}

```

Key Points: `_count++` is NOT atomic — it's three CPU operations (read/modify/write). Two threads can read the same value and both increment, losing an update. `lock(obj)` acquires a mutual exclusion lock — only one thread enters at a time. `Thread.Join()` waits for a thread to complete before continuing.

27. Simulate and Resolve a Deadlock

Objective: Create deadlock scenario; resolve with Monitor.TryEnter.

```
using System;
using System.Threading;

class DeadlockDemo
{
    private static readonly object LockA = new object();
    private static readonly object LockB = new object();

    // DEADLOCK SCENARIO (commented out – would freeze program)
    // Thread1: acquires A, waits for B
    // Thread2: acquires B, waits for A – DEADLOCK!

    // RESOLVED: use Monitor.TryEnter with timeout
    static void ThreadSafe1()
    {
        bool gotA = false, gotB = false;
        try
        {
            gotA = Monitor.TryEnter(LockA, TimeSpan.FromSeconds(1));
            if (!gotA) { Console.WriteLine("T1: Could not acquire A"); return; }
            Console.WriteLine("T1: Acquired lock A");
            Thread.Sleep(100);

            gotB = Monitor.TryEnter(LockB, TimeSpan.FromSeconds(1));
            if (!gotB) { Console.WriteLine("T1: Could not acquire B – releasing A"); return; }
            Console.WriteLine("T1: Acquired lock B");
            Console.WriteLine("T1: Working with both locks...");
        }
        finally
        {
            if (gotB) Monitor.Exit(LockB);
            if (gotA) Monitor.Exit(LockA);
        }
    }

    static void ThreadSafe2()
    {
        bool gotA = false, gotB = false;
        try
        {
            gotB = Monitor.TryEnter(LockB, TimeSpan.FromSeconds(1));
            if (!gotB) { Console.WriteLine("T2: Could not acquire B"); return; }
            Console.WriteLine("T2: Acquired lock B");
            Thread.Sleep(100);

            gotA = Monitor.TryEnter(LockA, TimeSpan.FromSeconds(1));
            if (!gotA) { Console.WriteLine("T2: Could not acquire A – releasing B"); return; }
            Console.WriteLine("T2: Acquired lock A");
            Console.WriteLine("T2: Working with both locks...");
        }
        finally
        {
            if (gotA) Monitor.Exit(LockA);
            if (gotB) Monitor.Exit(LockB);
        }
    }

    public static void Run()
    {
        Thread t1 = new Thread(ThreadSafe1);
        Thread t2 = new Thread(ThreadSafe2);
        t1.Start(); t2.Start();
        t1.Join(); t2.Join();
        Console.WriteLine("Both threads completed without deadlock.");
    }
}
```

```

    }
}

class Program
{
    static void Main() => DeadlockDemo.Run();
}

```

Key Points: Deadlock = Thread A holds Lock1 waiting for Lock2, Thread B holds Lock2 waiting for Lock1. Monitor.TryEnter(lock, timeout) returns false instead of blocking forever. Release acquired locks in finally to guarantee cleanup even on exceptions.

28. Log with System.Diagnostics.Trace

Objective: Log to console and file using Trace and TextWriterTraceListener.

```

using System;
using System.Diagnostics;
using System.IO;

class Logger
{
    private const string LogFile = "app_trace.log";

    public static void Initialize()
    {
        // Console listener
        Trace.Listeners.Add(new ConsoleTraceListener());

        // File listener
        var fileListener = new TextWriterTraceListener(LogFile);
        fileListener.TraceOutputOptions = TraceOptions.DateTime | TraceOptions.ThreadId;
        Trace.Listeners.Add(fileListener);

        Trace.AutoFlush = true; // write immediately (no buffer)
    }

    public static void Info(string message)
        => Trace.TraceInformation($"[INFO] {DateTime.Now:HH:mm:ss} - {message}");

    public static void Warning(string message)
        => Trace.TraceWarning($"[WARN] {DateTime.Now:HH:mm:ss} - {message}");

    public static void Error(string message)
        => Trace.TraceError($"[ERROR] {DateTime.Now:HH:mm:ss} - {message}");

    public static void CloseAll() => Trace.Close();
}

class Program
{
    static void Main()
    {
        Logger.Initialize();

        Logger.Info("Application started.");
        Logger.Info("Loading user data...");
        Logger.Warning("User count exceeds threshold (>1000).");
        Logger.Error("Database connection timeout on retry #3.");
        Logger.Info("Application shutting down.");

        Logger.CloseAll();
        Console.WriteLine($"
Log also saved to: {Path.GetFullPath("app_trace.log")}");
    }
}

```

Key Points: TraceInformation/Warning/Error map to severity levels. TextWriterTraceListener writes to any TextWriter (file, StringWriter). AutoFlush = true ensures no log entries are lost if the app crashes. Trace.Close() flushes and closes all listeners.

29. Sanitize Input and Prevent XSS

Objective: Sanitize user input using HTML encoding to prevent XSS injection.

```
using System;
using System.Net;
using System.Text.RegularExpressions;

class InputSanitizer
{
    // HTML encode: converts < > & " ' to safe HTML entities
    public static string HtmlEncode(string input)
    {
        if (string.IsNullOrEmpty(input)) return string.Empty;
        return WebUtility.HtmlEncode(input);
    }

    // Strip all HTML tags
    public static string StripHtml(string input)
    {
        if (string.IsNullOrEmpty(input)) return string.Empty;
        return Regex.Replace(input, "<.*?>", string.Empty);
    }

    // Sanitize for safe display: encode AND trim
    public static string Sanitize(string input)
    {
        if (string.IsNullOrWhiteSpace(input)) return string.Empty;
        string trimmed = input.Trim();
        string encoded = WebUtility.HtmlEncode(trimmed);
        return encoded;
    }
}

class UserForm
{
    public string Name { get; set; }
    public string Comment { get; set; }

    public void Submit(string rawName, string rawComment)
    {
        // Simulate receiving dangerous input
        Console.WriteLine("--- Raw Input ---");
        Console.WriteLine($"Name : {rawName}");
        Console.WriteLine($"Comment: {rawComment}");

        // Sanitize
        Name = InputSanitizer.Sanitize(rawName);
        Comment = InputSanitizer.Sanitize(rawComment);

        Console.WriteLine("
--- Sanitized (Safe to Display/Store) ---");
        Console.WriteLine($"Name : {Name}");
        Console.WriteLine($"Comment: {Comment}");
    }
}

class Program
{
    static void Main()
    {
        var form = new UserForm();

        // XSS attack attempts
        form.Submit(
            "<script>alert('XSS');</script>Alice",
            "<img src=x onerror='document.cookie'> Great event!"
        );
    }
}
```

Key Points: WebUtility.HtmlEncode converts < to < > to > making script tags harmless. Never display raw user input in HTML without encoding. Regex.Replace with <.*?> strips all HTML tags (non-greedy to avoid over-matching). Defense in depth: validate, sanitize, and encode at every layer.

[illegible]

[illegible]


```

        UpdateSalary(1, 82000);
        ReadAllEmployees();

        DeleteEmployee(2);
        ReadAllEmployees();

        // Department summary via DataAdapter
        var summary = GetDepartmentSummary();
        Console.WriteLine("
-- Department Summary --");
        foreach (DataRow row in summary.Rows)
            Console.WriteLine($"{row["Department"],-15} " +
                               $"Count={row["HeadCount"]} " +
                               $"AvgSalary={row["AvgSalary"]:N2}");
    }
}

```

Key Points: SqlConnection opens a database connection. SqlCommand executes SQL. Always use parameterized queries (@Name) to prevent SQL injection — never concatenate user input. ExecuteNonQuery for INSERT/UPDATE/DELETE. ExecuteReader for SELECT with streaming. SqlDataAdapter + DataTable loads all results into memory — good for small datasets. using statements ensure connections are properly closed and disposed.

End of C# and ADO.NET — All 30 Solutions Complete